



**UNIVERSIDAD TECNOLÓGICA
DE QUERÉTARO**

Alumna:

Vasquez Noyola Andrea

Grupo:

IDGS17

Matrícula:

2023148002

Materia:

Seguridad en el Desarrollo de Aplicaciones

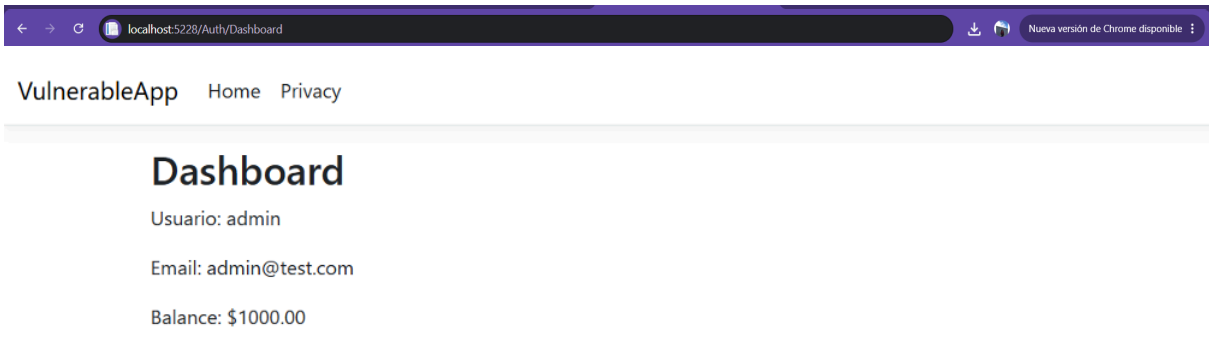
Profesor:

Gerardo Ramírez Villareal

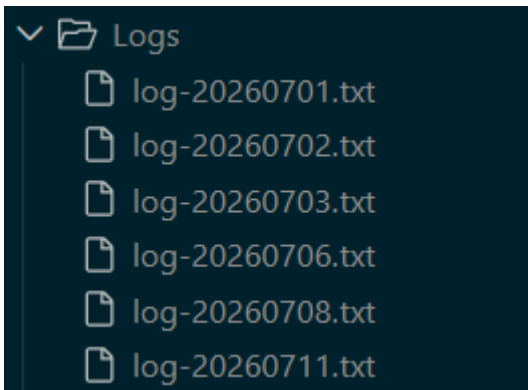
Práctica 4 Pt1

Evidencias

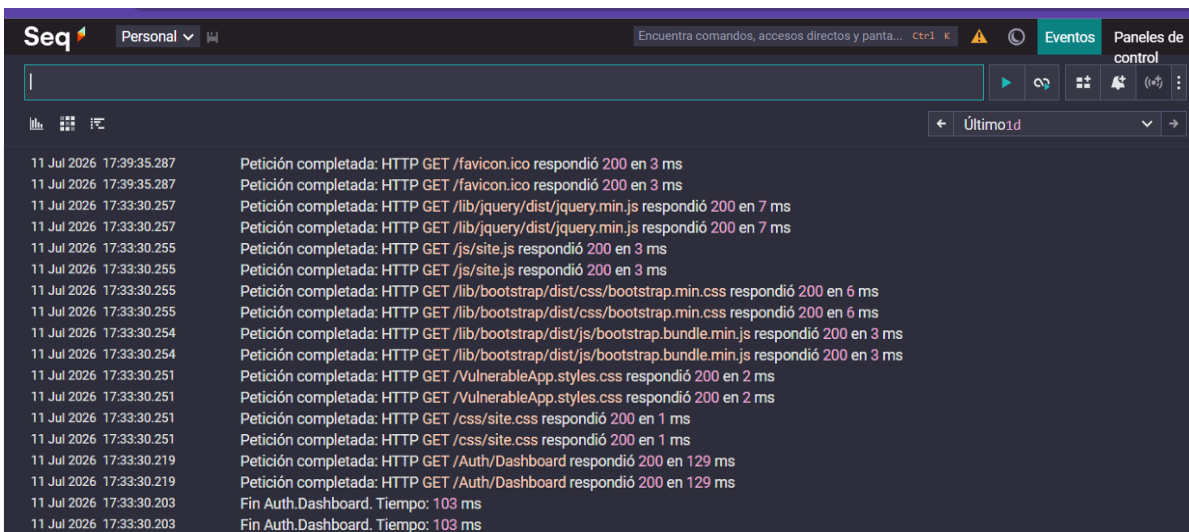
Captura de VulnerableApp generando registros.



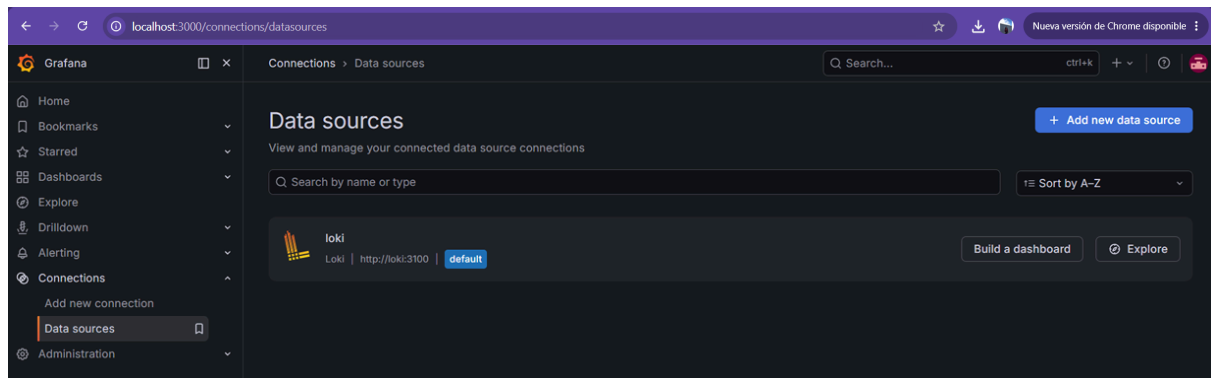
Captura de VulnerableApp generando registros.



Captura de Seq mostrando los eventos.



Captura de Grafana con Loki configurado.



Captura de contenedores en ejecución (docker ps)

PS C:\Users\Andyy\OneDrive\Documentos\SDA_8vo\practicas\vulnerableapp>

docker ps

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
9c66762207c3	grafana/protmail:latest	"/usr/bin/protmail -..."	4 minutes ago	Up 4 minutes	
8681f84b8e0c	grafana/loki:latest	loki "/usr/bin/loki -config..."	4 minutes ago	Up 4 minutes	

docker-compose.yml actualizado.

```
version: "3.9"

services:
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    ports:
      - "3000:3000"
    volumes:
      - grafana_data:/var/lib/grafana
    networks:
      - observability_net

  loki:
    image: grafana/loki:latest
    container_name: loki
    ports:
      - "3100:3100"
    volumes:
```

```

    - loki_data:/loki
networks:
    - observability_net

promtail:
    image: grafana/promtail:latest
    container_name: promtail
    volumes:
        # Bind Mount: Conecta la carpeta local de Logs al contenedor de
        # Promtail
        - ./Logs:/var/log/VulnerableApp
        # Monta el archivo de configuración que crearemos en el siguiente
        # paso
        - ./promtail-config.yaml:/etc/promtail/config.yaml
    command: -config.file=/etc/promtail/config.yaml
    networks:
        - observability_net

networks:
    observability_net:

volumes:
    grafana_data:
    loki_data:

```

Archivo de configuración de Promtail.

```

server:
    http_listen_port: 9080
    grpc_listen_port: 0

positions:
    filename: /tmp/positions.yaml

clients:
    - url: http://loki:3100/loki/api/v1/push

scrape_configs:
    - job_name: vulnerableapp_logs
      static_configs:
        - targets:
            - localhost
          labels:
            job: vulnerableapp

```

```

    app: VulnerableApp
    __path__: /var/log/VulnerableApp/*.txt

pipeline_stages:
  - regex:

      expression:
'.*(?P<keyword>Auth\.Login|Auth\.Dashboard|Search\.Index).*'
      - template:
          source: category

          template: '{{ if eq .keyword "Auth.Login" }}Security{{ else
if eq .keyword "Auth.Dashboard" }}Security{{ else if eq .keyword
"Search.Index" }}Audit{{ else }}General{{ end }}'
      - labels:
          category:

```

Reto y preguntas de reflexión

¿Qué modificaciones realizó en Promtail?

Se configuró el archivo promtail-config.yaml para integrar un pipeline de procesamiento de registros. Las modificaciones clave incluyen:

Definición de scrape_configs: Se estableció la ruta /var/log/VulnerableApp/*.txt para que Promtail monitoree los archivos de log generados por la aplicación mediante el volumen compartido (bind mount).

Pipeline de etiquetas dinámicas: Se añadieron pipeline_stages utilizando expresiones regulares (regex) para identificar palabras clave en el contenido de los logs (como Auth.Login o Search.Index).

Asignación de categorías: Se implementó una lógica de template que clasifica los eventos en etiquetas (labels) personalizadas como "Security" o "Audit" basándose en el tipo de actividad detectada, lo cual permite que Loki indexe estos registros de forma estructurada.

¿Cómo se utiliza Loki las Labels?

Loki utiliza las etiquetas (labels) para indexar los flujos de registros (log streams). A diferencia de sistemas de búsqueda de texto completo, Loki indexa solo los metadatos (las etiquetas). Esto permite que el sistema sea altamente eficiente: cuando realizas una consulta, Loki filtra primero por estas etiquetas para reducir el volumen de datos que debe procesar, y luego busca el texto dentro de esos flujos específicos.

¿Qué ventajas ofrecen para realizar consultas con LogQL?

El uso de etiquetas junto con LogQL (el lenguaje de consulta de Loki) ofrece las siguientes ventajas:

Velocidad: Permite realizar consultas rápidas al descartar grandes cantidades de datos que no coinciden con las etiquetas de interés.

Agrupación lógica: Facilita la creación de paneles en Grafana donde puedes agrupar registros por servicios, niveles de criticidad (error, info) o, como en tu caso, por categorías de negocio (Security/Audit), mejorando la legibilidad.

Consultas complejas: Permite realizar operaciones matemáticas o estadísticas sobre los logs (como contar eventos por categoría) de manera sencilla.

¿Qué ventajas ofrece incorporar una plataforma de observabilidad sin modificar la aplicación existente?

Esta arquitectura, basada en el desacoplamiento, ofrece beneficios críticos para sistemas en producción:

1. **Estabilidad y bajo riesgo:** Al no tocar el código fuente, se elimina la posibilidad de introducir errores (bugs) o afectar la lógica de negocio al intentar implementar el monitoreo.
2. **Evolución independiente:** La plataforma de observabilidad puede actualizarse, escalarse o reemplazarse (por ejemplo, cambiar Seq por Loki) sin necesidad de recompilar o desplegar nuevas versiones de la aplicación principal.
3. **Centralización y eficiencia:** Permite aplicar una estrategia de observabilidad uniforme a múltiples microservicios o aplicaciones sin importar el lenguaje en que fueron desarrollados, siempre que generen archivos de log. Esto facilita la evolución del sistema, ya que los equipos de operaciones pueden obtener visibilidad total del estado de salud del sistema, detectar cuellos de botella y auditar eventos de seguridad sin depender de los desarrolladores para añadir código de telemetría a cada nueva funcionalidad.